

Applying SOLID Principles in an AI-Assisted E-Commerce System

Problem Statement

You have joined a mid-sized product company that operates a multi-vendor e-commerce platform. The system is being upgraded to include an AI-assisted decision layer built using LangChain.

The AI layer is expected to assist in:

- Selecting optimal payment strategies.
- Recommending discount strategies.
- Personalizing notifications.
- Flagging potentially fraudulent orders.

The current implementation (simplified) is:

```
class OrderService {  
    void placeOrder(String customerName, double amount, String paymentType) {  
  
        if (paymentType.equals("CARD")) {  
            // process card payment via Razorpay  
  
        } else if (paymentType.equals("UPI")) {  
            // process UPI via PhonePe  
        }  
  
        // save order to MySQL database  
  
        // send email confirmation  
    }  
}
```

The AI pipeline is integrated as follows:

User Input → Prompt → LLM → Decision → OrderService

The LLM is responsible for:

- Payment method suggestion
- Discount recommendation
- Notification personalization

The system must also satisfy the following constraints:

1. Different vendors may require different payment gateways.

2. Notifications must support Email, SMS, and WhatsApp based on user preference.
3. Some vendors require audit logging for every transaction.
4. The system must support A/B testing of discount strategies.
5. The engineering team wants to avoid:
 - Over-engineering
 - Excessive class creation and abstraction
 - Performance overhead caused by excessive abstraction and AI latency.

Objectives and Requirements

Part A: Design Flaw Analysis

Analyze the existing implementation and:

- Identify three design flaws in the current system.
- Each flaw must be explained in the context of AI integration challenges.
- Each identified flaw must relate to at least one of the specified system constraints.

Part B: Refactored Architecture with AI Integration

Propose a system design integrating the backend with a LangChain-based AI pipeline.

Your design must include:

- Core backend services.
- LangChain-based AI pipeline.
- Key abstractions (AI decision interfaces, tools, services).
- Placement of the AI pipeline within the architecture.
- Components that should remain deterministic (non-AI) and justification.
- One trade-off where applying SOLID increases either:
 - latency, or
 - system complexity.

Part C: SOLID Principle Analysis

Given the following implementation:

```

class AIDecisionService {
    String decide(String input) {
        if(input.contains("payment")) return "CARD";
        else if(input.contains("discount")) return "10%";
    }
}

```

Explain:

- Which SOLID principle is violated.
- Why this violation is particularly harmful in an LLM-integrated system.

Part D: System Extension

Extend your previously proposed design to support:

- 1. Buy Now, Pay Later (BNPL)**
 - AI suggests BNPL eligibility.
 - External credit approval API.
 - Conditional eligibility checks before payment.
- 2. Push Notifications**
 - Only for mobile users.
 - AI selects Email/SMS/WhatsApp.
 - Deterministic fallback when AI confidence is low.
- 3. AI-Driven Festive Combo Discount**
 - Multiple AI-generated discount strategies.
 - A/B testing support.

Your design should explain:

- 1. Design Approach**
 - Where the AI pipeline is used.
 - Where deterministic logic overrides AI.
 - Which components remain unchanged.
- 2. SOLID Justification**

For each feature:

 - Identify the applicable SOLID principle(s).
 - Explain how the design prevents tight coupling with AI components.
- 3. Robustness Consideration**
 - Assume the LLM occasionally produces incorrect or inconsistent outputs.